



US012387286B2

(12) **United States Patent**
MacDonald et al.

(10) **Patent No.:** **US 12,387,286 B2**

(45) **Date of Patent:** **Aug. 12, 2025**

(54) **METHODS AND APPARATUS FOR
HARDWARE ACCELERATED IMAGE
PROCESSING FOR SPHERICAL
PROJECTIONS**

(71) Applicant: **GoPro, Inc.**, San Mateo, CA (US)

(72) Inventors: **William Edward MacDonald**, San
Diego, CA (US); **Kyler William
Schwartz**, Oceanside, CA (US); **David
A. Newman**, San Diego, CA (US)

(73) Assignee: **GoPro, Inc.**, San Mateo, CA (US)

(*) Notice: Subject to any disclaimer, the term of this
patent is extended or adjusted under 35
U.S.C. 154(b) by 0 days.

(21) Appl. No.: **18/405,646**

(22) Filed: **Jan. 5, 2024**

(65) **Prior Publication Data**

US 2024/0144414 A1 May 2, 2024

Related U.S. Application Data

(63) Continuation of application No. 17/930,602, filed on
Sep. 8, 2022, now Pat. No. 11,887,210, which is a
(Continued)

(51) **Int. Cl.**
G06T 1/20 (2006.01)
G06T 1/60 (2006.01)
(Continued)

(52) **U.S. Cl.**
CPC **G06T 1/20** (2013.01); **G06T 1/60**
(2013.01); **G06T 5/50** (2013.01); **G06T 5/70**
(2024.01); **G06T 2207/20221** (2013.01)

(58) **Field of Classification Search**
CPC G06T 1/20; G06T 1/60; G06T 5/50; G06T
5/70; G06T 2207/20221
See application file for complete search history.

(56) **References Cited**

U.S. PATENT DOCUMENTS

5,257,051 A 10/1993 Bushroe
5,703,701 A 12/1997 Yamamoto
(Continued)

FOREIGN PATENT DOCUMENTS

WO WO 2017176346 A1 * 10/2017 G06T 3/00
WO 2018236050 A1 12/2018

OTHER PUBLICATIONS

“3rd Generation Partnership Project; Technical Specification Group
Services and System Aspects; Virtual Reality (VR) media services
over 3GPP (Release 15)”, 3GPP Standard ; Technical Report ; 3GPP
TR 26.918, 3rd Generation Partnership Project (3GPP), Mobile
Competence Centre ; 650, Route Des Lucioles ; F-06921 Sophia-
Antipolis Cedex ; France, (May 3, 2017), vol. SA WG4, No. V0.7.0,
pp. 1-58, XP051298214.

(Continued)

Primary Examiner — Devona E Faulk

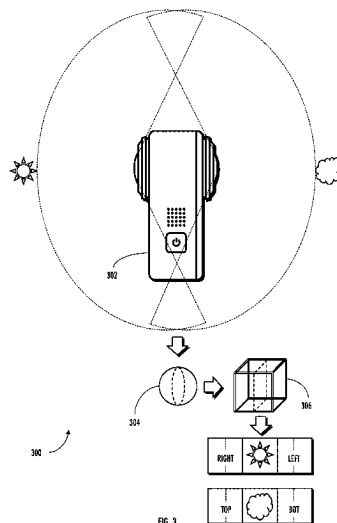
Assistant Examiner — Donna J. Ricks

(74) *Attorney, Agent, or Firm* — Wang Hardoon, P.C.

(57) **ABSTRACT**

Methods and apparatus for image processing of spherical
content via hardware acceleration components. In one
embodiment, an EAC image is subdivided into facets via
existing software addressing and written into the memory
buffers (normally used for rectilinear cubemaps) in a graph-
ics processing unit (GPU). The EAC facets may be trans-
lated, rotated, and/or mirrored so as to align with the
expected three-dimensional (3D) coordinate space. The
GPU may use existing hardware accelerator logic, paral-
lelization, and/or addressing logic to greatly improve 3D
image processing effects (such as a multi-band blend using
Gaussian blurs.)

20 Claims, 11 Drawing Sheets



Related U.S. Application Data

continuation of application No. 17/079,183, filed on Oct. 23, 2020, now Pat. No. 11,481,863.

(60) Provisional application No. 62/925,138, filed on Oct. 23, 2019.

(51) **Int. Cl.**
G06T 5/50 (2006.01)
G06T 5/70 (2024.01)

(56) **References Cited**

U.S. PATENT DOCUMENTS

6,118,556	A	9/2000	Yamamoto	2010/0271394	A1	10/2010	Howard
6,141,034	A	10/2000	McCutchen	2010/0299630	A1	11/2010	McCutchen
6,373,895	B2	4/2002	Saunders	2011/0010497	A1	1/2011	Bryant-Rich
7,835,437	B1	11/2010	Zhang	2011/0013849	A1	1/2011	Saito
8,055,974	B2	11/2011	Mizuno	2011/0135000	A1	6/2011	Alshina
8,606,029	B1	12/2013	Wong	2012/0086850	A1	4/2012	Irani
9,172,924	B1	10/2015	Martel	2012/0128058	A1	5/2012	Bakharov
9,294,757	B1	3/2016	Lewis	2012/0163725	A1	6/2012	Fukuhara
9,721,393	B1	8/2017	Dunn	2012/0194685	A1	8/2012	Kawakami
9,787,862	B1	10/2017	Newman	2012/0300984	A1	11/2012	Dann
9,894,422	B2	2/2018	Hwang	2013/0128121	A1	5/2013	Agarwala
10,187,568	B1	1/2019	Tran	2013/0141572	A1	6/2013	Torres
10,321,109	B1 *	6/2019	Tanumihardja H04N 9/641	2013/0155250	A1	6/2013	Myers
10,349,055	B1	7/2019	Waggoner	2013/0162766	A1	6/2013	Cohen
10,666,941	B1	5/2020	Doron	2013/0166580	A1	6/2013	Maharajh
11,004,176	B1	5/2021	Newman	2013/0195205	A1	8/2013	Wang
11,024,008	B1	6/2021	Newman	2013/0198233	A1	8/2013	Jacobson
11,049,219	B2	6/2021	Newman	2013/0198337	A1	8/2013	Nix
11,109,067	B2	8/2021	Newman	2014/0119675	A1	5/2014	Kim
11,228,781	B2	1/2022	Newman	2014/0133568	A1	5/2014	Otsuka
11,481,863	B2	10/2022	Macdonald	2014/0165125	A1	6/2014	Lee
11,790,488	B2	10/2023	Newman	2014/0192151	A1	7/2014	Wang
2001/0036356	A1	11/2001	Weaver	2014/0218608	A1	8/2014	Stelliga
2002/0009151	A1	1/2002	Gentric	2014/0267842	A1	9/2014	Lee
2002/0097322	A1	7/2002	Monroe	2014/0347450	A1	11/2014	Han
2002/0141659	A1	10/2002	Florin	2014/0348248	A1	11/2014	Ihara
2002/0159632	A1	10/2002	Chui	2015/0163476	A1	6/2015	Grafulla-González
2003/0016289	A1	1/2003	Motomura	2015/0187389	A1	7/2015	Horita
2003/0048921	A1	3/2003	Cahill	2015/0222293	A1	8/2015	Lida
2003/0103678	A1	6/2003	Hsuan	2015/0319462	A1	11/2015	Ramasubramonian
2003/0123857	A1	7/2003	Egawa	2015/0350682	A1	12/2015	Zhang
2003/0206582	A1	11/2003	Srinivasan	2016/0073023	A1	3/2016	Rondinelli
2004/0156548	A1	8/2004	Kodama	2016/0094847	A1	3/2016	Ibrahim
2004/0158878	A1	8/2004	Ratnakar	2016/0103715	A1	4/2016	Sethia
2004/0240856	A1	12/2004	Yahata	2016/0112489	A1	4/2016	Adams
2005/0044112	A1	2/2005	Yamamoto	2016/0134881	A1	5/2016	Wang
2005/0195899	A1	9/2005	Han	2016/0148431	A1	5/2016	Seydoux
2005/0265619	A1	12/2005	Ozaki	2016/0150241	A1	5/2016	Hirabayashi
2006/0001742	A1	1/2006	Park	2016/0165309	A1	6/2016	Van Brandenburg
2006/0024041	A1	2/2006	Lou	2016/0261373	A1	9/2016	Nagasaka
2006/0034523	A1	2/2006	Park	2016/0335484	A1	11/2016	Xie
2006/0256380	A1	11/2006	Klassen	2016/0337706	A1	11/2016	Hwang
2007/0005795	A1	1/2007	Gonzalez	2016/0360104	A1	12/2016	Zhang
2007/0065139	A1	3/2007	Ishii	2016/0360295	A1	12/2016	Hwang
2007/0102520	A1	5/2007	Carlson	2016/0373771	A1	12/2016	Hendry
2007/0253490	A1	11/2007	Makino	2017/0078574	A1	3/2017	Puntambekar
2007/0274599	A1	11/2007	Ishikawa	2017/0094278	A1	3/2017	Bickerstaff
2007/0291115	A1	12/2007	Bachelder	2017/0118540	A1	4/2017	Thomas
2008/0084926	A1	4/2008	Valenzuela	2017/0148186	A1	5/2017	Holzer
2008/0101455	A1	5/2008	Scheelke	2017/0155924	A1	6/2017	Gokhale
2008/0129844	A1	6/2008	Cusack	2017/0195565	A1	7/2017	Ollier
2008/0159594	A1	7/2008	Chiu	2017/0214861	A1	7/2017	Rachlin
2008/0225132	A1	9/2008	Inaguma	2017/0236252	A1	8/2017	Nguyen
2008/0252719	A1	10/2008	Choi	2017/0243384	A1	8/2017	Huang
2008/0259409	A1	10/2008	Nishizawa	2017/0272758	A1	9/2017	Lin
2009/0219282	A1	9/2009	Kim	2017/0295373	A1	10/2017	Zhang
2010/0053212	A1	3/2010	Kang	2017/0302990	A1	10/2017	Ying
2010/0071010	A1	3/2010	Elnathan	2017/0336705	A1	11/2017	Zhou
2010/0088646	A1	4/2010	Nishimori	2017/0339341	A1	11/2017	Zhou
2010/0157018	A1	6/2010	Lampotang	2017/0366808	A1	12/2017	Lin
2010/0157090	A1	6/2010	Kobayashi	2018/0027178	A1	1/2018	Macmillan
2010/0169779	A1	7/2010	Mason	2018/0027181	A1	1/2018	Roulet
				2018/0027226	A1 *	1/2018	Abbas G06T 15/20
							348/47
				2018/0035047	A1	2/2018	Lei
				2018/0040164	A1	2/2018	Newman
				2018/0054695	A1	2/2018	Castor
				2018/0075635	A1	3/2018	Choi
				2018/0101931	A1	4/2018	Abbas
				2018/0103199	A1	4/2018	Hendry
				2018/0160160	A1	6/2018	Swaminathan
				2018/0184121	A1	6/2018	Kim
				2018/0190323	A1	7/2018	De Jong
				2018/0192074	A1	7/2018	Shih
				2018/0218484	A1	8/2018	Shen
				2018/0276800	A1	9/2018	Abbas
				2018/0288356	A1	10/2018	Ray
				2018/0343470	A1	11/2018	Schmit
				2018/0352245	A1	12/2018	Macdonald

(56)

References Cited**U.S. PATENT DOCUMENTS**

2019/0007669	A1	1/2019	Kim	
2019/0007676	A1	1/2019	Katsumata	
2019/0035064	A1	1/2019	Franklin	
2019/0104326	A1	4/2019	Stockhammer	
2019/0158859	A1	5/2019	Zhang	
2019/0213425	A1	7/2019	Anderson	
2019/0222756	A1	7/2019	Moloney	
2019/0230283	A1	7/2019	Ollier	
2019/0235748	A1	8/2019	Seol	
2019/0313026	A1	10/2019	Cheng	
2019/0333237	A1	10/2019	Javidnia	
2020/0014954	A1	1/2020	Watson	
2020/0053392	A1	2/2020	Hannuksela	
2020/0053393	A1	2/2020	Niamut	
2020/0084428	A1	3/2020	Oh	
2020/0092582	A1	3/2020	Xiu	
2020/0107003	A1	4/2020	Phillips	
2020/0250454	A1	8/2020	Croxford	
2020/0351449	A1	11/2020	Oh	
2021/0192796	A1	6/2021	Aflaki Beni	
2021/0392375	A1	12/2021	Newman	
2022/0060738	A1	2/2022	Newman	
2022/0132144	A1*	4/2022	Sauer	H04N 19/597
2023/0011843	A1	1/2023	Macdonald	

OTHER PUBLICATIONS

Hall; GPU Accelerated Feature Algorithms for Mobile Devices; Mar. 2014; School of Computing and Mathematical Sciences, Auckland University; pp. 1-150.

International Search Report and Written Opinion issued in PCT/US2019/68386, dated Mar. 20, 2020, 11 pages.

Jeppsson et al. "Efficient Live and on-Demand Tiled HEVC 360 VR Video Streaming," 2018 IEEE International Symposium on Multimedia (ISM), Taichung, 2018, pp. 81-88.

Paul Bourke; Converting a fisheye image into a panoramic, spherical or perspective projection; Jul. 2016 (25 pages).

Qin et al., "On-Demand Sharing of a High-Resolution Panorama Video from Networked Robotic Cameras", Proceedings of the 2007 IEEE/RSJ International Conference on Intelligent Robots and Systems San Diego, CA, USA, Oct. 29-Nov. 2, 2007, 6 pages.

Yuwen He et al, "JVET-D0021 (DOC). AHG8: InterDigital's projection format conversion tool", Joint Video Exploration Team (JVET). of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11. 4th Meeting: Chengdu, CN, Oct. 15-21, 2016, (Oct. 21, 2016), URL: http://phenix.int-evry.fr/jvet/doc_end_user/current_document.php?id=2717, (Oct. 29, 2019), XP055637250, 35 pages.

* cited by examiner

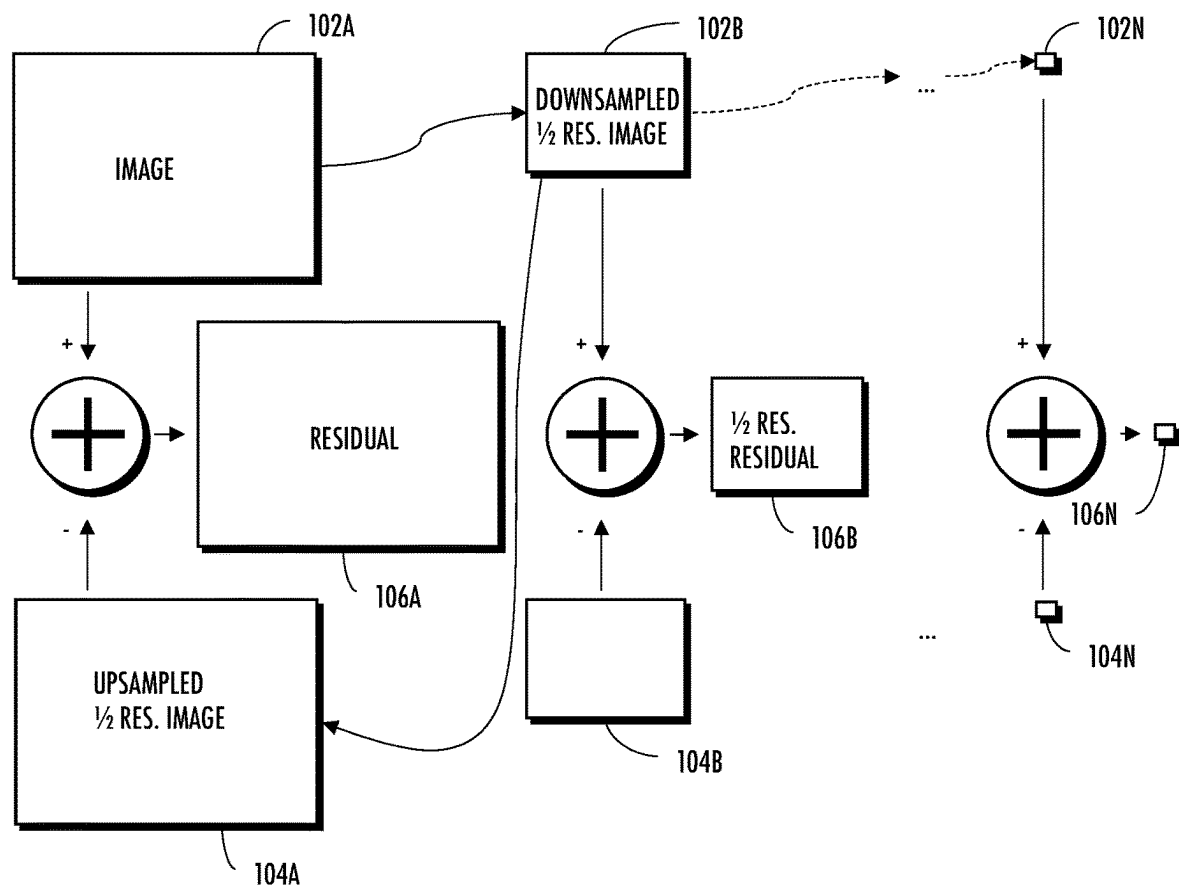


FIG. 1A

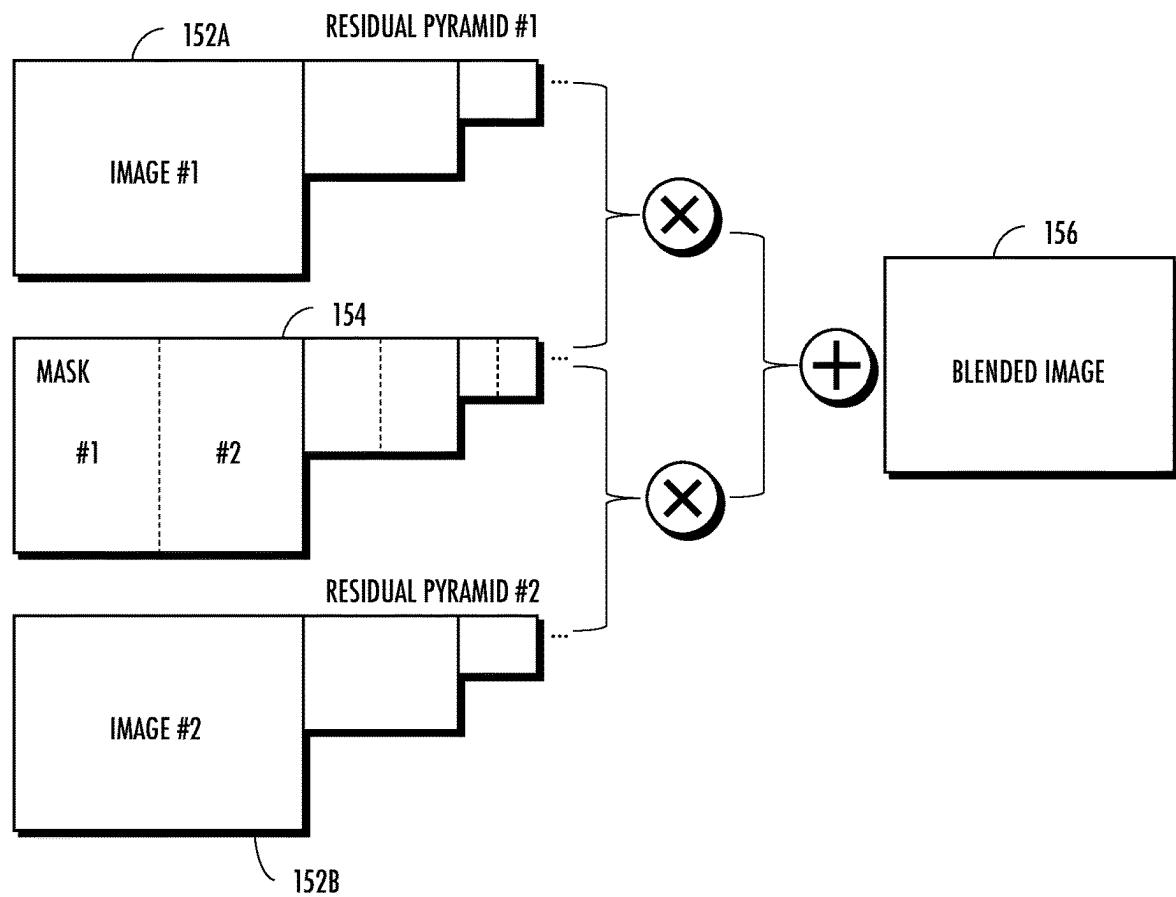


FIG. 1B

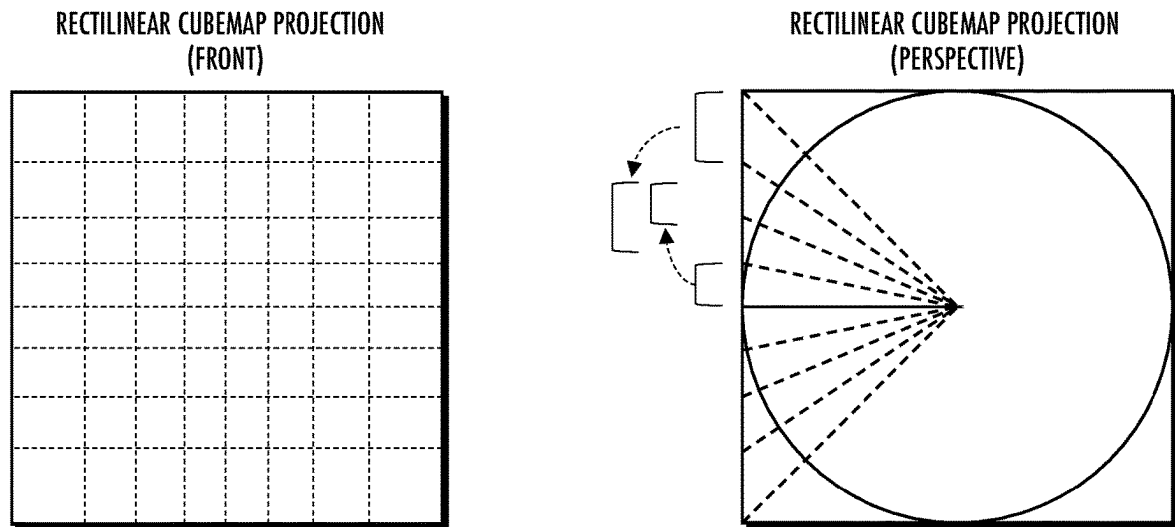


FIG. 2A

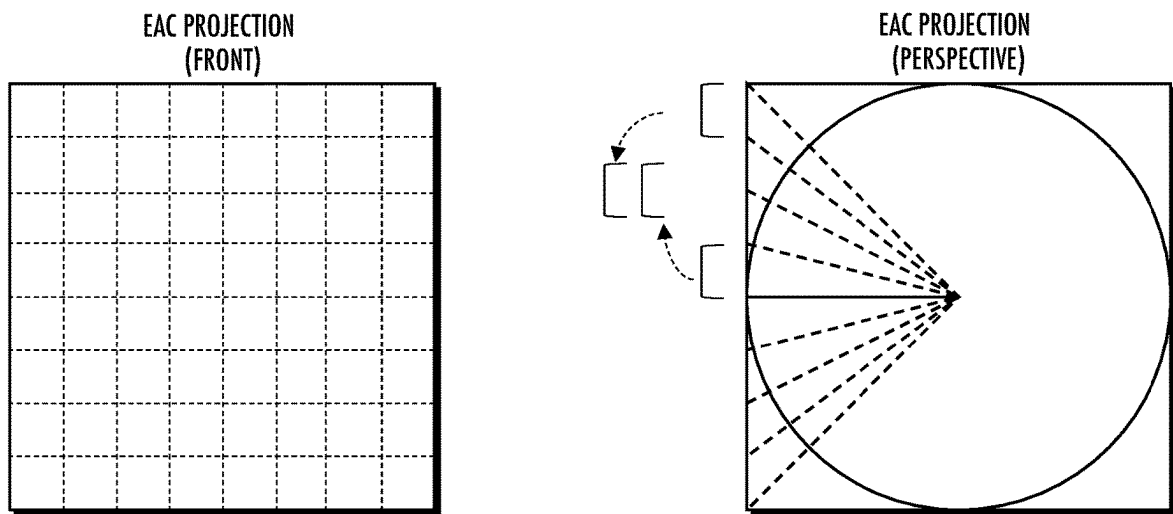


FIG. 2B

EAC DATA STRUCTURE

LEFT	FRONT	RIGHT
TOP	BACK	BOTTOM



RECTILINEAR CUBEMAP DATA STRUCTURE

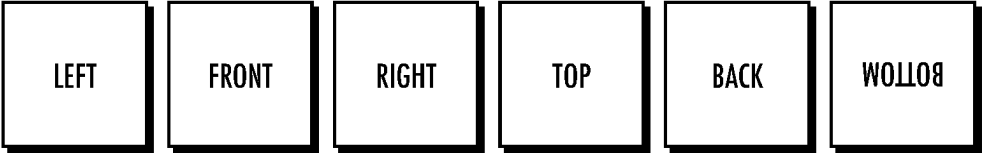
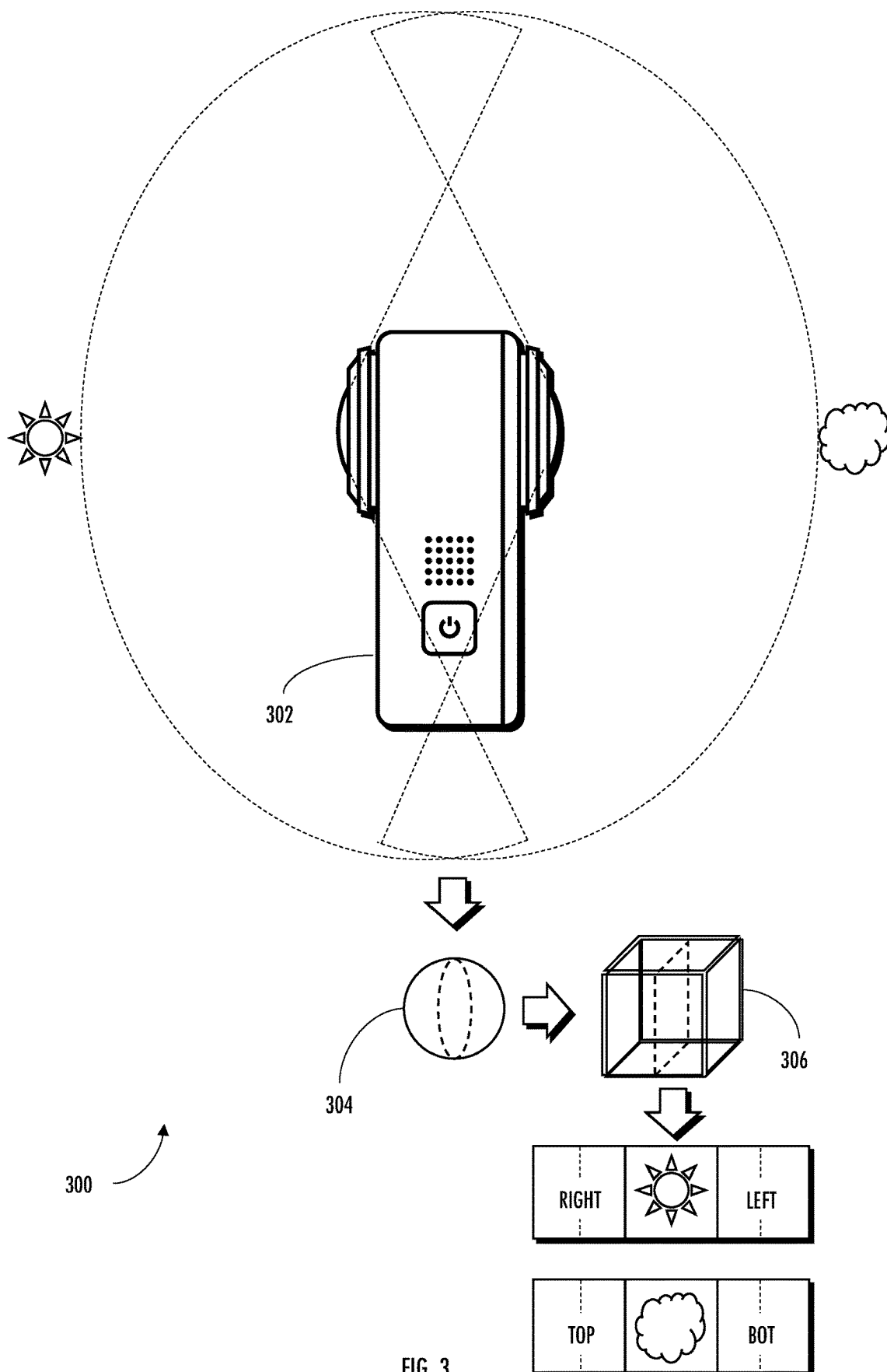


FIG. 2C



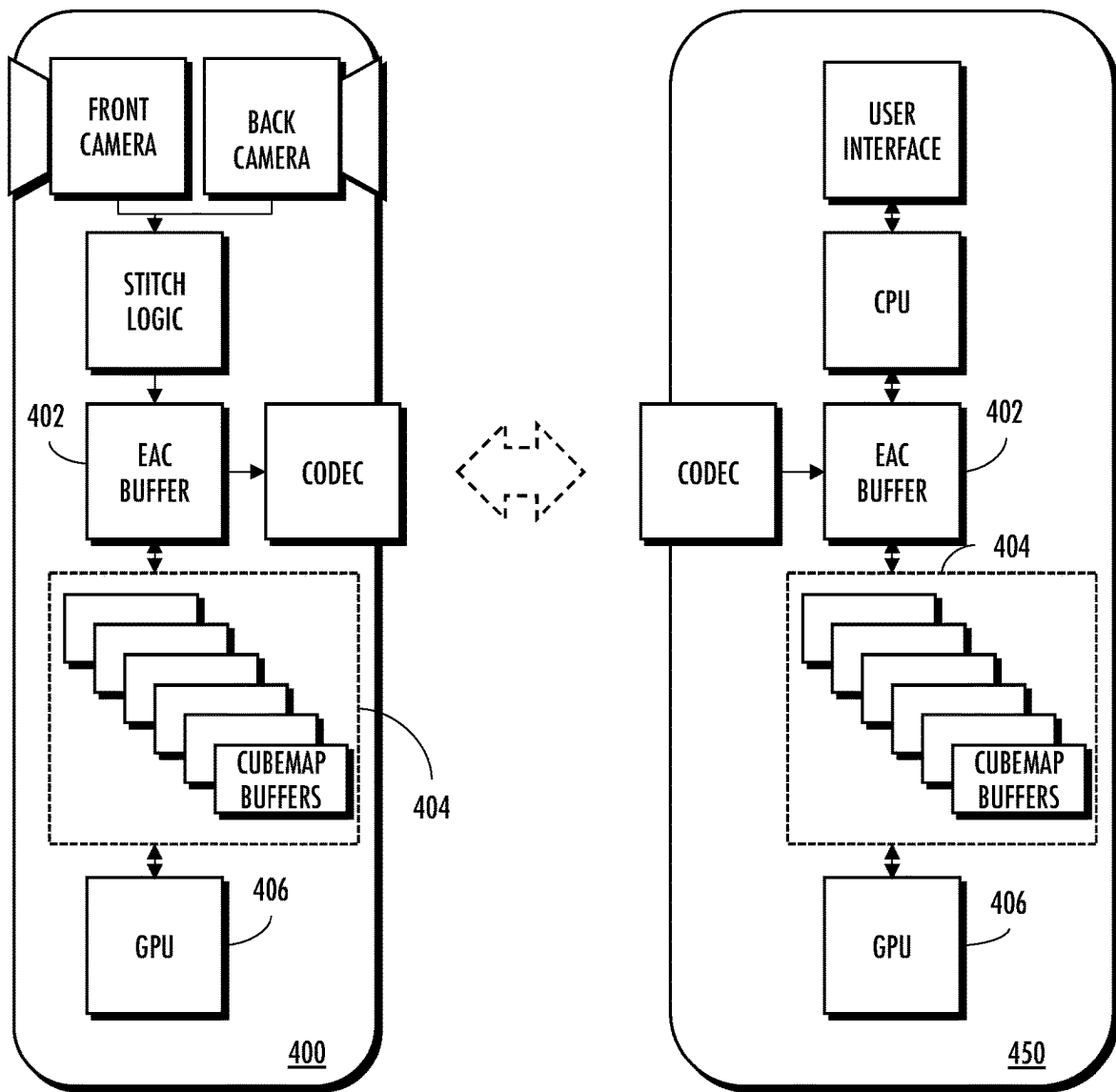


FIG. 4A

EAC DATA STRUCTURE

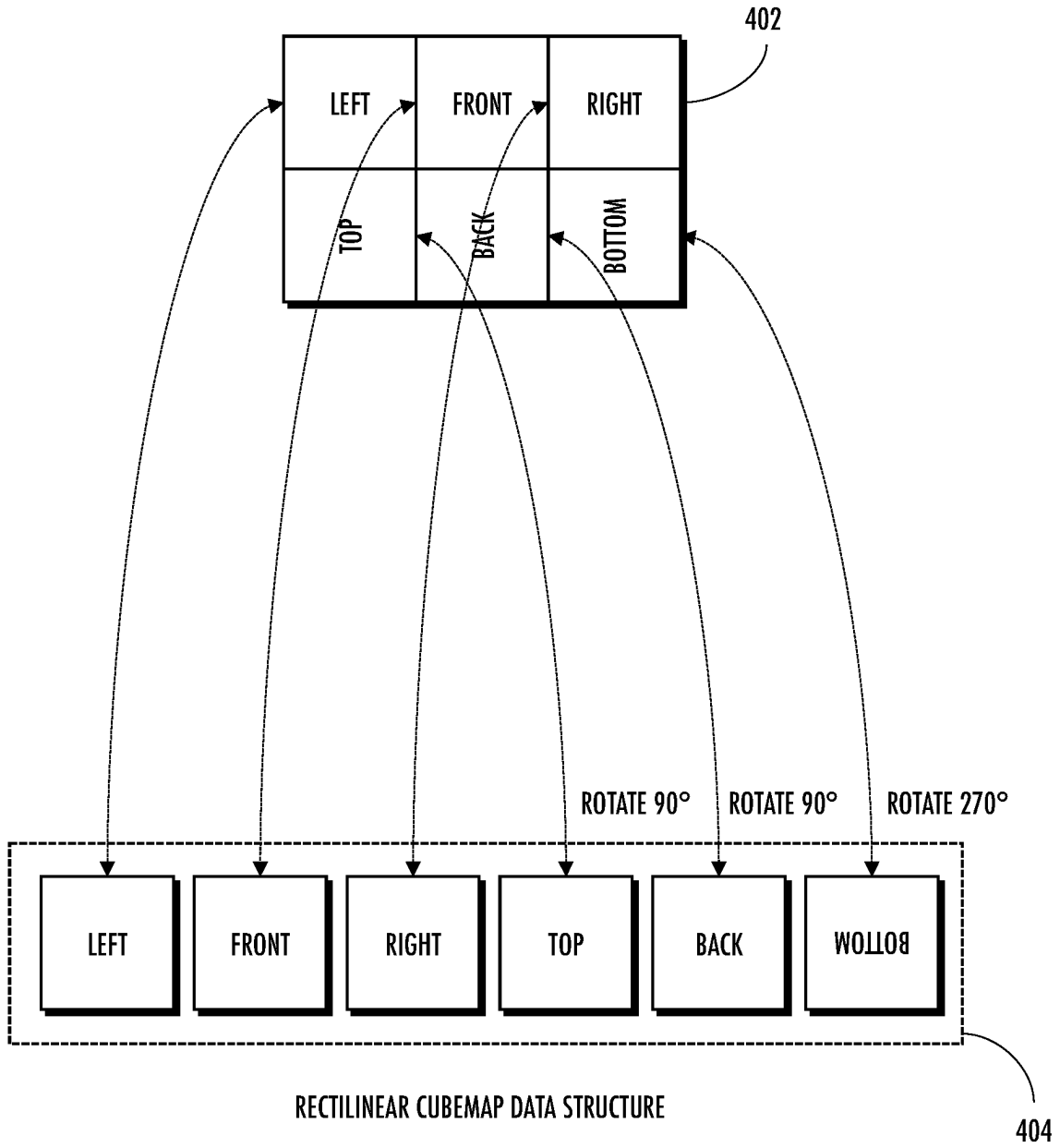
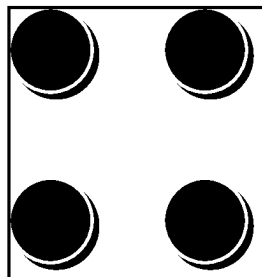


FIG. 4B

LINEAR
BLUR



3X3
GAUSSIAN
BLUR

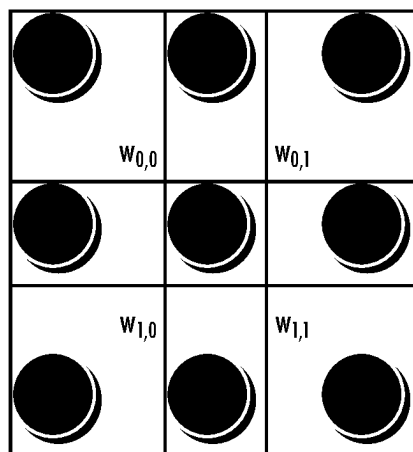


FIG. 5A

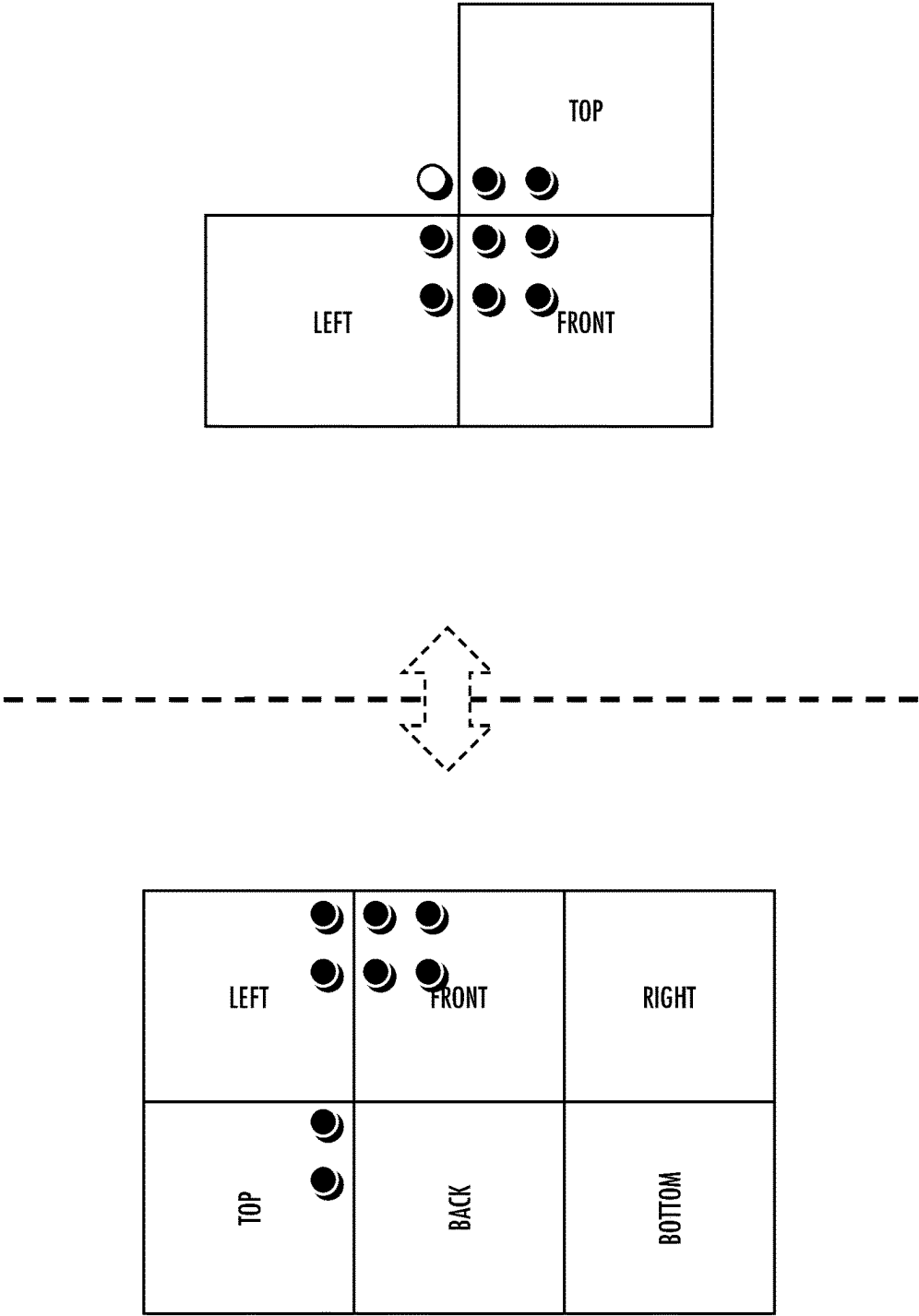


FIG. 5B

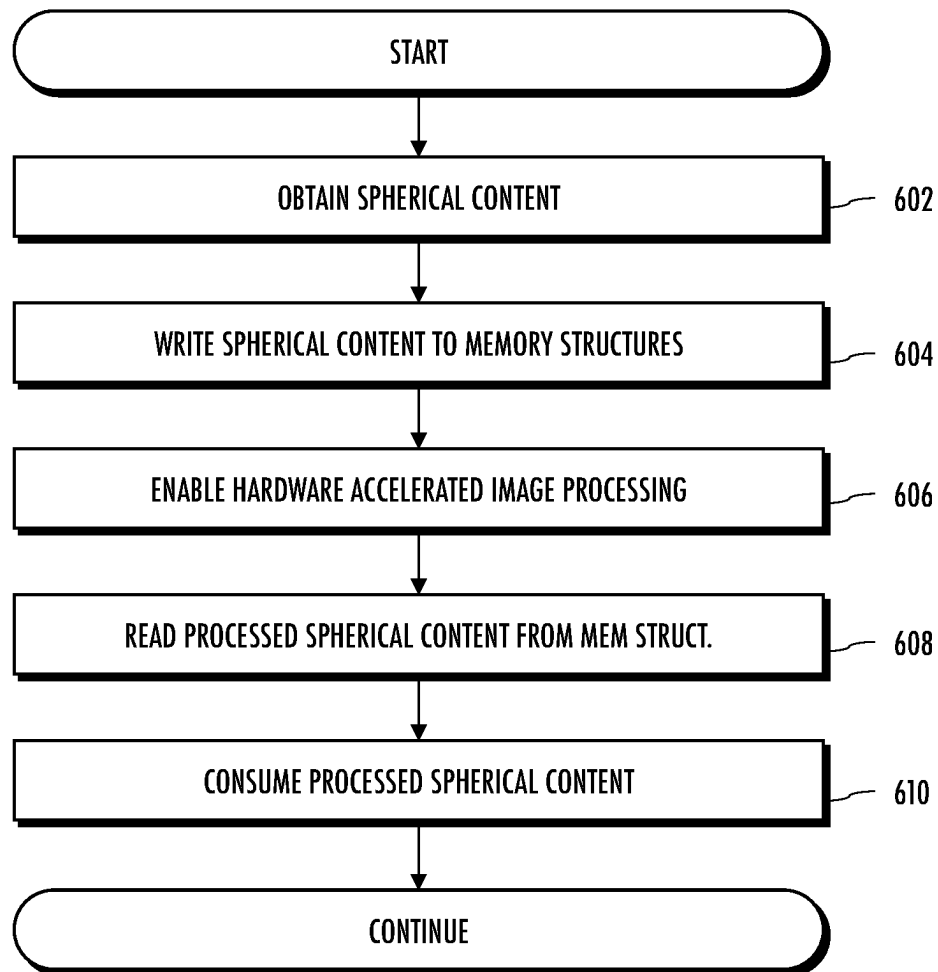


FIG. 6

1

METHODS AND APPARATUS FOR HARDWARE ACCELERATED IMAGE PROCESSING FOR SPHERICAL PROJECTIONS

PRIORITY

This application is a continuation of, and claims the benefit of priority to U.S. patent application Ser. No. 17/930,602 filed Sep. 8, 2022 and entitled “METHODS AND APPARATUS FOR HARDWARE ACCELERATED IMAGE PROCESSING FOR SPHERICAL PROJECTIONS”, that claims the benefit of priority to U.S. patent application Ser. No. 17/079,183 filed Oct. 23, 2020 and entitled “METHODS AND APPARATUS FOR HARDWARE ACCELERATED IMAGE PROCESSING FOR SPHERICAL PROJECTIONS”, that claims the benefit of priority to U.S. Provisional Patent Application Ser. No. 62/925,138 filed Oct. 23, 2019 and entitled “METHODS AND APPARATUS FOR HARDWARE ACCELERATED IMAGE PROCESSING FOR SPHERICAL PROJECTIONS”, each of which are incorporated by reference in its entirety.

RELATED APPLICATIONS

This application is generally related to the subject matter of U.S. patent application Ser. No. 16/572,383 filed Sep. 16, 2019 and entitled “METHODS AND APPARATUS FOR MAXIMIZING CODEC BANDWIDTH IN VIDEO APPLICATIONS,” incorporated herein by reference in its entirety.

COPYRIGHT

A portion of the disclosure of this patent document contains material that is subject to copyright protection. The copyright owner has no objection to the facsimile reproduction by anyone of the patent document or the patent disclosure, as it appears in the Patent and Trademark Office patent files or records, but otherwise reserves all copyright rights whatsoever.

TECHNICAL FIELD

This disclosure relates to image processing. Specifically, the present disclosure relates in one exemplary aspect to image processing of spherical content within embedded systems (e.g., processor, memory, and/or battery constrained consumer electronics devices).

DESCRIPTION OF RELATED TECHNOLOGY

Sophisticated image processing techniques can be used to achieve different visual effects. As but one such example, “multi-band blending” is a commonly used technique for combining images. Advantageously, multi-band blending preserves high frequency image information (e.g., edges are not washed out), while still ensuring that low frequency image information is smoothly blended.

Unfortunately, due to the high memory and computational complexity requirements, sophisticated image processing has historically been relegated to resource intensive environments, e.g., post-processing workstations and/or specialized video game hardware acceleration. Existing solutions are not desirable for embedded device manufacturers (such as the GoPro HERO™ or Fusion™ families of devices

2

manufactured by the Assignee hereof) or their ecosystems (e.g., smart phones, laptops, and/or other mobile media playback devices).

BRIEF DESCRIPTION OF THE DRAWINGS

The disclosed embodiments have other advantages and features which will be more readily apparent from the following detailed description of the invention and the appended claims, when taken in conjunction with the accompanying drawings.

FIG. 1A is a graphical representation of “pyramid” image processing, useful in explaining multi-band blending.

FIG. 1B is a graphical representation of multi-band blending based on pyramid image representations, useful in explaining various embodiments of the present disclosure.

FIGS. 2A-2C are graphical representations of rectilinear cubemaps and equi-angular cubemaps (EAC), useful in explaining various embodiments of the present disclosure.

FIG. 3 is a logical block diagram illustrating one exemplary approach for capturing spherical content, useful in explaining various embodiments of the present disclosure.

FIG. 4A is a logical block diagram of an action camera in data communication with a smart phone, useful to illustrate various aspects of the present disclosure.

FIG. 4B is a graphical representation of an exemplary memory translation operation between a first memory buffer and a second memory buffer, useful to illustrate various aspects of the present disclosure.

FIG. 5A is a graphical representation of one exemplary method for approximating a Gaussian blur with hardware accelerated linear blur logic, useful to illustrate various aspects of the present disclosure.

FIG. 5B is a graphical representation of hardware accelerated addressing logic, useful to illustrate various aspects of the present disclosure.

FIG. 6 is a logical block diagram of one exemplary method for image processing spherical content via hardware acceleration components, consistent with various principles of the present disclosure.

DETAILED DESCRIPTION

In the following detailed description, reference is made to the accompanying drawings which form a part hereof, wherein like numerals designate like parts throughout, and in which is shown, by way of illustration, embodiments that may be practiced. It is to be understood that other embodiments may be utilized, and structural or logical changes may be made without departing from the scope of the present disclosure. Therefore, the following detailed description is not to be taken in a limiting sense, and the scope of embodiments is defined by the appended claims and their equivalents.

Aspects of the disclosure are disclosed in the accompanying description. Alternate embodiments of the present disclosure and their equivalents may be devised without departing from the spirit or scope of the present disclosure. It should be noted that any discussion herein regarding “one embodiment”, “an embodiment”, “an exemplary embodiment”, and the like indicate that the embodiment described may include a particular feature, structure, or characteristic, and that such particular feature, structure, or characteristic may not necessarily be included in every embodiment. In addition, references to the foregoing do not necessarily comprise a reference to the same embodiment. Finally, irrespective of whether it is explicitly described, one of

3

ordinary skill in the art would readily appreciate that each of the particular features, structures, or characteristics of the given embodiments may be utilized in connection or combination with those of any other embodiment discussed herein.

Various operations may be described as multiple discrete actions or operations in turn, in a manner that is most helpful in understanding the claimed subject matter. However, the order of description should not be construed as to imply that these operations are necessarily order dependent. In particular, these operations may not be performed in the order of presentation. Operations described may be performed in a different order than the described embodiment. Various additional operations may be performed and/or described operations may be omitted in additional embodiments.

Existing Multi-Band Image Blending

As a brief aside, there are many different image processing techniques that may be used to blend images; one commonly used technique is so-called “multi-band blending.”

FIG. 1A is a graphical representation of “pyramid” image processing useful in explaining multi-band blending. As shown in FIG. 1A, an image can be separated into its constituent frequency components via an iterative process of blurring and subtraction. This is analogous to a two-dimensional wavelet transform (a decomposition of a signal into a set of contractions, expansions, and translations of a basis wavelet). For example, an image **102A** may be downsampled to a half-resolution image **102B**. The half-resolution image **102B** is then upsampled to create an upsampled W resolution image **104A**. The residual image **106A** is created by subtracting the upsampled image **104A** from the original image **102A**. This process is iteratively repeated to the extent required (each iteration requires more processing resources but preserves fidelity across a broader frequency range). Notably, the residual “pyramid” of images **106A** through **106N** contains all of the difference information necessary to recreate the original image **102A** from the downsampled counterpart **102N** (which may be a single pixel for images having evenly divisible dimensions).

Referring now to FIG. 1B, one representation of multi-band blending based on pyramid image representations is shown. As shown therein, multi-band blending can be performed with two images by building residual pyramid image representations **152A**, **152B** of the two images. A “mask” pyramid **154** is constructed. The mask **154** defines a blend weight between the first and the second image pyramids for each pyramid level. For each pyramid level, the levels of the two pyramids **152A**, **152B** are combined based on the weights of the mask pyramid **154** and summed together. Collapsing together all the level-wise summations, generates a resulting output blended image **156**.

Many variants of multi-band blending exist for a variety of different applications and/or constraints. For example, some implementations may only use a subset of the image pyramid tiers, to reduce processing and/or memory burden (resulting in some image information loss). Similarly, some implementations may use different masking weights; for example, linear weights and gaussian weights. Yet other variants may adjust how upsampling and/or downsampling are performed; for example, “non-power-of-two” images cannot be evenly halved (downsampled); a variety of techniques exist to handle the remainders.

Equi-Angular (EAC) Cubemap Versus Rectilinear Cubemap Projections

Recently, the growing popularity of action cameras and other wide-angle field of view (FOV) applications have

4

elevated the importance of three-dimensional (3D) image data structures. Two of the most popular projections are equi-angular cubemap (EAC) and rectilinear cubemap.

FIG. 2A is a graphical representation of frontal and perspective projection according to a rectilinear cubemap. A rectilinear cubemap projects an image onto a flat surface using a radial projection. As used herein, the term “rectilinear” refers to a visual perspective based on an idealized pin-hole camera at the center of the cube. As shown in the perspective view of FIG. 2A; object size and/or geometry distortion in the image is a function of angular displacement. In other words, rectilinear cubemaps are characterized by a substantial variation in pixel to image information density (pixels/area).

FIG. 2B is a graphical representation of frontal and perspective projection according to an equi-angular cubemap (EAC). Unlike rectilinear cubemaps, the EAC projection projects an image onto a flat surface keeping the amount of pixels constant per unit of angular displacement. EAC does not preserve rectilinearity but does ensure that pixel to image information density (pixels/area) is consistent throughout the entire image.

For further reference, a mathematical conversion between rectilinear cubemap and EAC is provided in a pseudocode snippet, included below:

```

01:  #define QPI 0.7853981634
02:  #define INV_QPI 1.273239545
03:
30:  vec3 DoEAC(vec3 xyz)
05:  {
06:      float maxAxis, x, y;
07:      vec3 absXYZ = abs(xyz);
08:      float signx = sign(xyz.x);
09:      float signy = sign(xyz.y);
35:      float signz = sign(xyz.z);
11:
12:      maxAxis = max(absXYZ.x, absXYZ.y);
13:      maxAxis = max(maxAxis, absXYZ.z);
14:
40:      float maxX = when_eq(absXYZ.x, maxAxis);
16:      float maxY = when_eq(absXYZ.y, maxAxis);
17:      float maxZ = when_eq(absXYZ.z, maxAxis);
18:
19:      x = -xyz.z*signx*maxX + xyz.x*maxY + xyz.x*signz*maxZ;
20:      y = xyz.y*maxX + -xyz.z*signy*maxY + xyz.y*maxZ;
21:
22:      // -1 to 1
23:      x = x / maxAxis;
24:      y = y / maxAxis;
25:
26:      //Undo EAC
27:      //x = INV_QPI * atan(x);
28:      //y = INV_QPI * atan(y);
50:
29:
30:      //OR
31:
32:      //Add EAC
33:      //x = tan(QPI * x);
34:      //y = tan(QPI * y);
55:
35:      xyz.x = signx*maxX + x*maxY + x*signz*maxZ;
36:      xyz.y = y*maxX + signy*maxY + y*maxZ;
37:      xyz.z = -x*signx*maxX + -y*signy*maxY + signz*maxZ;
39:
60:      return xyz;
41:  }

```

Pseudocode Snippet

FIG. 2C is a graphical illustration of the different data structures used to represent rectilinear cubemaps and equi-

5

angular cubemaps (EAC). As shown therein, the EAC is a two-dimensional (2D) array of pixel values. Notably, the EAC data structure is designed for delivery via commodity codecs and/or legacy image formats (e.g., MPEG A/V containers); existing software addressing techniques for EAC in three-dimensional (3D) coordinate space are typically only used to pack and unpack the image during transfer between devices or playback (seldomly performed). The image data is also rotated to minimize compression/decompression artifacts (e.g., left, front, right faces are oriented in the same direction; top, back, bottom faces are oriented in the same direction).

In contrast, rectilinear cubemaps are composed of six (6) distinct two-dimensional (2D) arrays of pixel values (an array of arrays). The rectilinear cubemap data structure originated in video game applications which require intensive image processing. Rectilinear cubemaps can be stored in dedicated hardware specific buffers according to relationships and rotations that have been optimized for graphics processing units (GPUs). For example, even though the cube faces are distinct data structures, GPU hardware-based wrapping logic is designed to treat adjacent pixels (e.g., from top and front faces) as neighbors.

As a brief aside, many video games rely on a “skybox” (a rectilinear cubemap that creates the illusion of distant three-dimensional (3D) surroundings). Notably, rectilinear cubemaps can be quickly displayed, but require significant amounts of memory (e.g., ~191% extra pixels relative to an EAC of the same imagery). More generally, video game consoles are designed to maximize player experience which is why rectilinear cubemaps are preferred; embedded systems operate under a fundamentally different design paradigm which is focused on minimizing memory footprints (as exemplified in the EAC data structure).

Existing Graphics Processing Units (GPUs)

GPUs have a number of characteristics that are functionally different from general purpose processing. As but one example, GPUs are constructed in a highly parallel structure for processing large blocks of image data in parallel (e.g., each pixel of the image may be similarly processed); in contrast, general purpose processing is generalized to handle a much wider variety of processing (e.g., frequent branching and/or conditional operations accessing many different types of data structures). GPUs additionally often incorporate specialized hardware for commonly used operations in image processing; examples of such hardware are e.g., dedicated linear blurring logic. As previously alluded to, GPUs may also include dedicated hardware for addressing rectilinear cubemaps in three-dimensional (3D) coordinate space. Moreover, GPUs provide a plethora of different edge behavior addressing variants that are commonly used in image processing applications (wrapping, clamping, mirroring, etc.)

While GPUs were originally optimized for video game consoles, the “bleed-through” of applications across different consumer electronics devices has resulted in a wide range of devices that include GPUs; commodity GPUs can now be found in embedded camera systems, mobile phones, personal computers, workstations, in addition to video game consoles.

Image Processing Spherical Content for Embedded Systems

FIG. 3 illustrates one approach of using two fisheye lenses in a back-to-back configuration **300** to generate a 360° panoramic image, common in so-called “action cameras.” In one such implementation, each lens covers more than a hemisphere (e.g., on the order of 190°). As shown therein, a 360° panorama is captured by an action camera **302** in two

6

(2) hyper-hemispherical images with overlapping regions. The action camera **302** stitches together a single spherical projection **304** based on the overlapping regions. The spherical projection **304** may be formatted into an EAC projection **306** for e.g., delivery via commodity codecs and/or image formats such as is described in U.S. patent application Ser. No. 16/572,383 filed Sep. 16, 2019 and entitled “METHODS AND APPARATUS FOR MAXIMIZING CODEC BANDWIDTH IN VIDEO APPLICATIONS,” incorporated above.

Notably, action cameras and other 360° applications have unique use considerations that are different than either e.g., traditional photography or video game applications. For example, the action camera **302** has two (2) different lighting conditions: the front camera is well lit, whereas the back camera is poorly lit. The differences in exposure between the front and back camera of FIG. 3 result in unique image processing complications. Specifically, the different lighting conditions for each camera sensor result in different exposure times and shutter speeds which affect color, white balance, and contrast in unpredictable ways. These differences in shooting conditions can create a visible “exposure transition” in the resulting images.

Ideally, the exposure transition can be blended using the aforementioned multi-band blending techniques. Unfortunately, existing image processing alternatives are poorly suited for embedded camera systems and/or other embedded devices (such as mobile phones).

Image processing spherical content in native EAC image formats is undesirable. The EAC data structure (see FIG. 2C above) is designed to minimize memory footprint for e.g., bulk transport via commodity codecs, and there are no GPUs that natively process EAC formats. Instead, EAC image processing must be handled in software emulation which is slow and power hungry. Software addressing has to repeatedly identify edge continuities in the EAC image array to implement different types of wrapping. And as noted before, general purpose processors are neither parallelized nor specialized for image processing operations.

Similarly, converting an EAC image to a rectilinear cubemap to leverage existing 2D image processing GPU techniques would be impractical. Modifying the post-captured image to a “skybox” would be computationally expensive and memory inefficient (~191% larger). Additionally, video game skyboxes are viewed as drawn; action cameras capture a 3D environment which may require extensive image processing. For example, multi-band blending 3D images requires Gaussian blurs. Further complicating matters, the substantial variation in pixel to image information density (pixels/area) at the edges of rectilinear cubemaps would result in undesirable underweighting of center pixels in Gaussian blurs; this results in noticeable visual artifacts.

Furthermore, embedded camera systems are commonly used on-the-go within an ecosystem of other embedded consumer electronics devices. Even if an embedded camera system could be specifically designed to handle spherical image processing, other embedded devices would not. For example, mobile phones suffer the same limitations; empirically, handling 3D post-processing on mobile phones is undesirably slow, and in some cases, prohibitively so.

To these ends, methods and apparatus are needed to, inter alia, enable sophisticated image processing techniques in embedded device platforms.

Example Operation

FIG. 4A is a logical block diagram of an action camera **400** in data communication with a smart phone **450**, useful

to illustrate various aspects of the present disclosure. While the illustrated embodiment is presented in the context of two (2) devices, the various techniques described herein may be performed by either in isolation.

In one exemplary embodiment, the action camera **400** captures two (2) hyper-hemispherical images with overlapping regions. The action camera **400** stitches together a single spherical projection based on the overlapping regions. The spherical projection is stored as an EAC projection into the first memory buffer **402**. The EAC projection uses software addressing techniques for EAC in three-dimensional (3D) coordinate space to write each EAC cube face to a second memory buffer **404** that is tightly coupled to a GPU **406**. Subsequently thereafter, the GPU **406** is configured to perform three-dimensional (3D) multi-band blending (Gaussian blurring, described hereinafter) to correct for “exposure transitions” (color, white balance, and contrast) caused by different lighting conditions between the front and rear camera. The resulting corrected EAC cube faces are written back to the first memory buffer **402** for transfer off-camera (e.g., to exemplary embedded device **450**).

FIG. **4B** is a graphical representation of the memory translation operation between the first memory buffer **402** and the second memory buffer **404**. As shown therein, the left, front, and right faces can be directly written. For example, the processor can use software based addressing to transfer the left, front, and right EAC faces to the corresponding left, front, and right buffers of the second memory buffer **404**. The top, back, and bottom EAC faces are rotated to match the GPU’s **406** hardware addressing. In one such implementation, the processor can use software based addressing to read the top and back faces, rotate the top and back faces by 90°, and store the top and back faces to the second memory buffer **404**. Similarly, the processor can use software based addressing to read the bottom face, rotate the bottom face by 270°, and store the bottom face to the second memory buffer **404**. Notably, the EAC image data is preserved in the translation; the image data stored in the second memory buffer **404** are equi-angular faces, not rectilinear faces.

Referring back to FIG. **4A**, in a second exemplary embodiment, an exemplary embedded device **450** receives spherical content via its codec and writes the content into a first memory buffer **402**. In some situations, the device **450** may be required to perform an image processing operation (e.g., multi-band blending). Under such circumstances, the device **450** uses software addressing techniques to write the spherical content to the second memory buffer **404**. Subsequently thereafter, the GPU **406** can perform the required image processing. The resulting output may be written back to the first memory buffer **402** for consumption thereon.

More directly, various embodiments of the present disclosure perform software translation of the EAC data structure (one (1) 2D array) to the native data structures traditionally used to store rectilinear cubemap faces (six (6) 2D arrays) to leverage existing GPU hardware for 3D image processing. Unlike rectilinear cubemaps, the resulting outputs are translated back to the EAC data structure format for use in the existing EAC ecosystem (e.g., display, transfer, etc.) Advantageously, processing EAC images in this manner leverages existing GPU advantages of e.g., hardware accelerated operations, hardware accelerated addressing, and high parallelization of image data processing.

As a related benefit, the commodity nature of GPU hardware across a variety of embedded devices enables sophisticated 3D image post-processing heretofore not thought possible. In particular, the aggressive processing

and/or memory limitations of embedded devices (such as smart phones and personal media devices) could not handle software emulated image processing techniques; as a practical matter, 3D image post-processing has been historically handled within high end desktop processors and/or workstations. The various techniques described herein not only make such image processing possible on embedded devices but do so on commodity components that are already widely deployed and in use.

Gaussian Blurring with Linear Blurs and Hardware Address Acceleration

As previously alluded to, multi-band blending may be used in conjunction with a variety of different blending techniques. Historically, GPUs provide linear blurring for blurring neighboring pixels of a two-dimensional (2D) image. Video game processing uses blurring to emulate a variety of different in-game effects e.g., motion blur, translucency, etc. However, video games seldom require (if at all) any 3D image processing of the skybox, since it was drawn or rendered ahead of time in accordance with the player’s intended viewpoint.

Existing 3D image processing techniques for smoothing panoramic multiple camera captures are based on Gaussian blurring. A Gaussian blur (also known as Gaussian smoothing) blurs an image by a Gaussian function to reduce image noise and reduce detail, while still preserving salient edge features. Gaussian blurring closely matches natural vision processing in the human perception system and enables smooth transitions that are visually palatable.

Arithmetically, Gaussian blurring can be approximated as a plurality of linear blurs of different weight. For example, as shown in FIG. **5A**, a 3×3 Gaussian blur can be implemented as four (4) linear blurs, each having a weight ($w_{0,0}$, $w_{0,1}$, $w_{1,0}$, $w_{1,1}$) corresponding to spatial location within the blur. Notably, Gaussian blurs may be of N×N dimension, the 3×3 implementation is purely an illustrative example. Handling Gaussian blurring with GPU linear blurring hardware acceleration (rather than in software emulation) provides substantial benefits both in power and memory as well as massive parallelization over the entire 2D EAC face.

As previously alluded to, in traditional EAC formats, every pixel of the EAC data structure has to be checked for wrapping behavior in order to properly perform a Gaussian blur. For example, as shown in FIG. **5B**, the 3×3 Gaussian blur is shown (not to scale) spanning the edges of the top-left-front EAC corner; two pixels (top face) are located in a wholly unrelated location and orientation of the EAC data structure. For a 3×3 Gaussian blur, nine (9) checks are required; for an N×N Gaussian blur, N² software address checks are required. The processing overhead for correct wrapping behavior is extraordinarily burdensome on embedded devices.

In contrast, various embodiments of the present disclosure greatly improve performance because the GPU handles hardware addressing natively across the EAC cube faces (no software addressing is required at all). There is no translation delay for blurs that span edges; the GPU hardware correctly treats adjacent pixels from different faces as neighbors. Similarly, existing GPU hardware accelerators handle non-power-of-two remainders for off-size displays (such as are common in wide angle FOV formats) via internal memory structures or rounding approximations.

Exemplary Methods

FIG. **6** is a logical block diagram of one exemplary method for image processing of spherical content via hardware acceleration components, consistent with various principles of the present disclosure.

At step 602 of the method, a device obtains spherical content. In one exemplary embodiment, the device includes multiple cameras that capture portions of spherical content which are then stitched together. In other embodiments, the device may receive spherical content that requires hardware accelerated image post-processing. In one exemplary embodiment, the spherical content (or image data for the spherical content) includes a three-dimensional (3D) equi-angular cubemap (EAC), or may be projected into an EAC image projection.

At step 604 of the method, the spherical content is written into memory structures. In one embodiment, the spherical content is subdivided and written into memory structures corresponding to a three-dimensional (3D) representation of image space. In one exemplary embodiment, an EAC image is subdivided into facets via existing software addressing and written into the memory buffers (normally used for rectilinear cubemaps) in a GPU. In one such variant, the EAC facets may be translated, rotated, and/or mirrored.

At step 606 of the method, hardware accelerated image processing is performed. In one embodiment, the hardware accelerated image processing configures 2D hardware acceleration to perform 3D image modifications. In one exemplary embodiment, GPU linear blur components (typically used in 2D space) are configured to approximate Gaussian blurs in 3D space. In one such variant, the linear blur components are distinctly weighted based on spatial coordinates (e.g., (0,0), (0,1), (1,0), (1,1), etc.) and blur size.

In one embodiment, wrapping can be performed using hardware accelerated address logic. In one exemplary embodiment, EAC wrapping leverages existing GPU addressing.

In one embodiment, upsampling and/or downsampling of non-power-of-two image content stores non-power-of-two remainders in dedicated GPU registers (or other GPU memory components). In one embodiment, upsampling and/or downsampling of non-power-of-two image content handles non-power-of-two remainders with arithmetic rounding approximations.

At step 608 of the method, the processed spherical content is read from the memory structures, and at step 610 of the method, the processed spherical content is consumed. In one embodiment, post-processed EAC content can be reconstructed from rectilinear cubemap buffers of a GPU. In one exemplary embodiment, the post-processed EAC content may be transferred to another device. In another exemplary embodiment, the post-processed EAC content may be displayed.

ADDITIONAL CONFIGURATION CONSIDERATIONS

Throughout this specification, some embodiments have used the expressions “comprises,” “comprising,” “includes,” “including,” “has,” “having” or any other variation thereof, all of which are intended to cover a non-exclusive inclusion. For example, a process, method, article, or apparatus that comprises a list of elements is not necessarily limited to only those elements but may include other elements not expressly listed or inherent to such process, method, article, or apparatus.

In addition, use of the “a” or “an” are employed to describe elements and components of the embodiments herein. This is done merely for convenience and to give a general sense of the invention. This description should be

read to include one or at least one and the singular also includes the plural unless it is obvious that it is meant otherwise.

As used herein any reference to any of “one embodiment” or “an embodiment”, “one variant” or “a variant”, and “one implementation” or “an implementation” means that a particular element, feature, structure, or characteristic described in connection with the embodiment, variant or implementation is included in at least one embodiment, variant or implementation. The appearances of such phrases in various places in the specification are not necessarily all referring to the same embodiment, variant or implementation.

As used herein, the term “computer program” or “software” is meant to include any sequence of human or machine cognizable steps which perform a function. Such program may be rendered in virtually any programming language or environment including, for example, Python, JavaScript, Java, C #/C++, C, Go/Golang, R, Swift, PHP, Dart, Kotlin, MATLAB, Perl, Ruby, Rust, Scala, and the like.

As used herein, the terms “integrated circuit”, is meant to refer to an electronic circuit manufactured by the patterned diffusion of trace elements into the surface of a thin substrate of semiconductor material. By way of non-limiting example, integrated circuits may include field programmable gate arrays (e.g., FPGAs), a programmable logic device (PLD), reconfigurable computer fabrics (RCFs), systems on a chip (SoC), application-specific integrated circuits (ASICs), and/or other types of integrated circuits.

As used herein, the term “memory” includes any type of integrated circuit or other storage device adapted for storing digital data including, without limitation, ROM, PROM, EEPROM, DRAM, Mobile DRAM, SDRAM, DDR/2 SDRAM, EDO/FPMS, RDRAM, SRAM, “flash” memory (e.g., NAND/NOR), memristor memory, and PSRAM.

As used herein, the term “processing unit” is meant generally to include digital processing devices. By way of non-limiting example, digital processing devices may include one or more of digital signal processors (DSPs), reduced instruction set computers (RISC), general-purpose (CISC) processors, microprocessors, gate arrays (e.g., field programmable gate arrays (FPGAs)), PLDs, reconfigurable computer fabrics (RCFs), array processors, secure microprocessors, application-specific integrated circuits (ASICs), and/or other digital processing devices. Such digital processors may be contained on a single unitary IC die or distributed across multiple components.

As used herein, the terms “camera” or “image capture device” may be used to refer without limitation to any imaging device or sensor configured to capture, record, and/or convey still and/or video imagery, which may be sensitive to visible parts of the electromagnetic spectrum and/or invisible parts of the electromagnetic spectrum (e.g., infrared, ultraviolet), and/or other energy (e.g., pressure waves).

Upon reading this disclosure, those of skill in the art will appreciate still additional alternative structural and functional designs as disclosed from the principles herein. Thus, while particular embodiments and applications have been illustrated and described, it is to be understood that the disclosed embodiments are not limited to the precise construction and components disclosed herein. Various modifications, changes and variations, which will be apparent to those skilled in the art, may be made in the arrangement, operation and details of the method and apparatus disclosed herein without departing from the spirit and scope defined in the appended claims.

11

It will be recognized that while certain aspects of the technology are described in terms of a specific sequence of steps of a method, these descriptions are only illustrative of the broader methods of the disclosure and may be modified as required by the particular application. Certain steps may be rendered unnecessary or optional under certain circumstances. Additionally, certain steps or functionality may be added to the disclosed implementations, or the order of performance of two or more steps permuted. All such variations are considered to be encompassed within the disclosure disclosed and claimed herein.

While the above detailed description has shown, described, and pointed out novel features of the disclosure as applied to various implementations, it will be understood that various omissions, substitutions, and changes in the form and details of the device or process illustrated may be made by those skilled in the art without departing from the disclosure. The foregoing description is of the best mode presently contemplated of carrying out the principles of the disclosure. This description is in no way meant to be limiting, but rather should be taken as illustrative of the general principles of the technology. The scope of the disclosure should be determined with reference to the claims.

It will be appreciated that the various ones of the foregoing aspects of the present disclosure, or any parts or functions thereof, may be implemented using hardware, software, firmware, tangible, and non-transitory computer-readable or computer usable storage media having instructions stored thereon, or a combination thereof, and may be implemented in one or more computer systems.

It will be apparent to those skilled in the art that various modifications and variations can be made in the disclosed embodiments of the disclosed device and associated methods without departing from the spirit or scope of the disclosure. Thus, it is intended that the present disclosure covers the modifications and variations of the embodiments disclosed above provided that the modifications and variations come within the scope of any claims and their equivalents.

What is claimed is:

1. A method for image processing of spherical content via hardware acceleration components, the method comprising:
 capturing a plurality of content portions from a plurality of image-capture apparatus;
 stitching the plurality of content portions into stitched content;
 generating an equi-angular cubemap projection from the stitched content;
 writing the spherical content to one or more memory structures, where the equi-angular cubemap projection comprises a first set of equi-angular image facets corresponding to a first set of the one or more memory structures and a second set of equi-angular image facets corresponding to a second set of the one or more memory structures;
 enabling a hardware accelerated image processor coupled to the one or more memory structures, where the hardware accelerated image processor addresses the first set of the one or more memory structures and the second set of the one or more memory structures in three-dimensional (3D) coordinate space;
 reading processed spherical content from the one or more memory structures; and
 consuming the processed spherical content.

12

2. The method of claim 1, further comprising causing the hardware accelerated image processor to approximate a Gaussian blur with a plurality of spatially weighted linear blurs.

3. The method of claim 1, where writing the spherical content to the one or more memory structures comprises at least one image transformation of at least a portion of the spherical content.

4. The method of claim 1, where blurring a portion of the spherical content treats certain pixels of the one or more memory structures as neighbor pixels.

5. The method of claim 1, where the processed spherical content is transferred to an other device.

6. An apparatus, comprising:

a plurality of buffers coupled to a graphics processing unit, where the graphics processing unit comprises a hardware accelerated addressing mechanism for the plurality of buffers that addresses a first set of the plurality of buffers and a second set of the plurality of buffers in three-dimensional (3D) coordinate space;

a processor; and

a non-transitory computer-readable apparatus comprising a plurality of instructions configured to, when executed by the processor, cause the apparatus to:

obtain a first image and write at least a first portion of the first image to the first set of the plurality of buffers;

obtain a second image and write at least a second portion of the second image to the second set of the plurality of buffers, where the first image comprises a first set of equi-angular image facets and the second image comprises a second set of equi-angular image facets; and

cause the graphics processing unit to blend the first portion and the second portion to create a blended portion.

7. The apparatus of claim 6, where the first portion of the first image is translated from a first two-dimensional image format to the first set of the plurality of buffers and the second portion of the second image is translated from a second two-dimensional image format to the second set of the plurality of buffers.

8. The apparatus of claim 7, where the hardware accelerated addressing mechanism further comprises wrapping logic that is configured to treat adjacent pixels as neighbor pixels.

9. The apparatus of claim 7, where the hardware accelerated addressing mechanism further comprises sampling logic that is configured to handle non-power-of-two remainders.

10. The apparatus of claim 7, where the apparatus further comprises a first camera configured to capture the first image in the first two-dimensional image format and a second camera configured to capture the second image in the second two-dimensional image format.

11. The apparatus of claim 7, where the apparatus further comprises a display and the blended portion and at least a portion of the first image or the second image is displayed to a user in a third two-dimensional image format.

12. An apparatus, comprising:

a plurality of buffers coupled to a graphics processing unit, where the graphics processing unit comprises a hardware accelerated addressing mechanism for the plurality of buffers that addresses a first set of the plurality of buffers and a second set of the plurality of buffers in three-dimensional (3D) coordinate space;

a processor; and

13

a non-transitory computer-readable apparatus comprising a plurality of instructions configured to, when executed by the processor, cause the apparatus to:
 obtain spherical content that comprises a set of equi-
 angular image facets stored as a two-dimensional data structure;

write at least a first portion of the spherical content to the first set of the plurality of buffers;

write at least a second portion of the spherical content to the second set of the plurality of buffers; and

cause the graphics processing unit to blend the first set of the plurality of buffers and the second set of the plurality of buffers to create blended image data.

13. The apparatus of claim **12**, where the hardware accelerated addressing mechanism further comprises wrapping logic that is configured to treat adjacent pixels as neighbor pixels.

14. The apparatus of claim **12**, where the hardware accelerated addressing mechanism further comprises sampling logic that is configured to handle non-power-of-two remainders.

15. The apparatus of claim **12**, where the apparatus further comprises logic to stitch the spherical content from two hyper-hemispherical images.

14

16. The apparatus of claim **12**, where the graphics processing unit approximates a Gaussian blur with a plurality of spatially weighted linear blurs.

17. The apparatus of claim **12**, where the plurality of instructions are further configured to, cause the apparatus to perform at least one image transformation of at least the first portion or the second portion.

18. The apparatus of claim **12**, where the plurality of instructions are further configured to, cause the apparatus to transfer the blended image data to an other device.

19. The apparatus of claim **12**, where the first portion is translated from a first two-dimensional image format to the first set of the plurality of buffers and the second portion is translated from a second two-dimensional image format to the second set of the plurality of buffers.

20. The apparatus of claim **19**, where the apparatus further comprises a first camera configured to capture a first image in the first two-dimensional image format and a second camera configured to capture a second image in the second two-dimensional image format.

* * * * *